

iSYST AKADEMIE

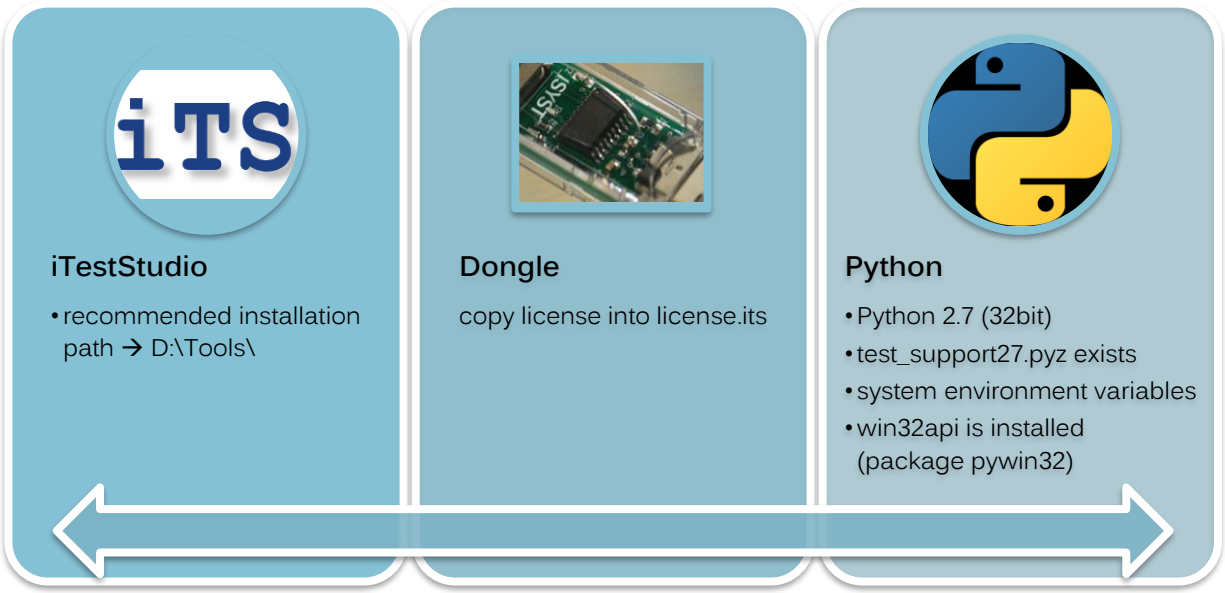
iTestStudio – Basic Level – Workshop

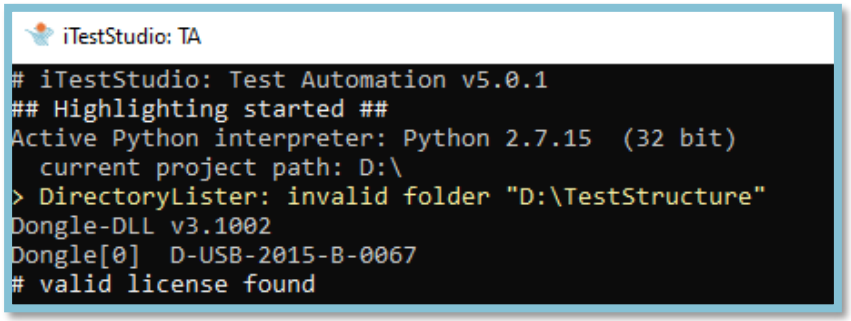
Preparations & Technology Check-In

to start the workshop

Getting started

What do we need to work with iTestStudio





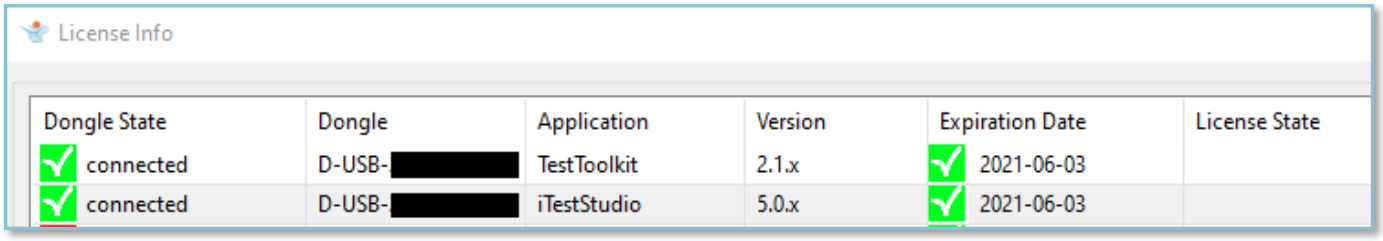
Getting started: iTestStudio

Dongle license

If not alredy don, copy your license code into **license.its**. (.../iTestStudio)

Open iTestStudio → click **Help** → **Show license information**

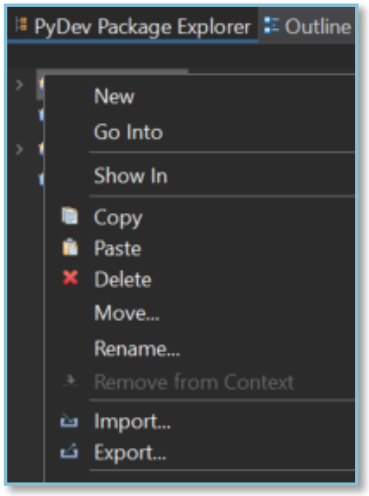
You should see green check marks now:

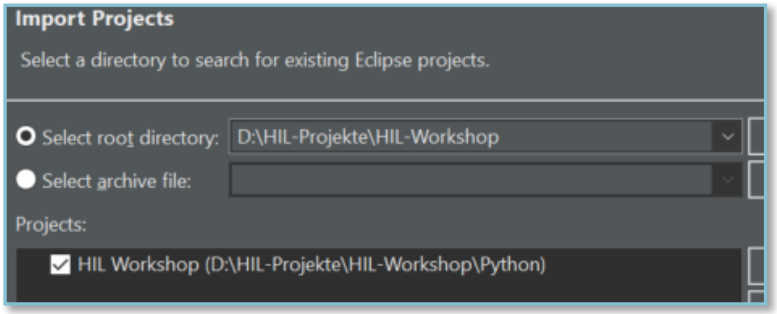


Getting started: TestEnvironment

Import Project into Eclipse 1/2

1	Copy the HIL-Workshop folder into D:\HIL-Projekte\
2	Open Eclipse and right-click in **PyDev Package Explorer** and select **Import...**
3	Click **Existing Projects into Workspace** and select the HIL-Workshop project





Getting started TestEnvironment

Import Project into Eclipse 2/2

4

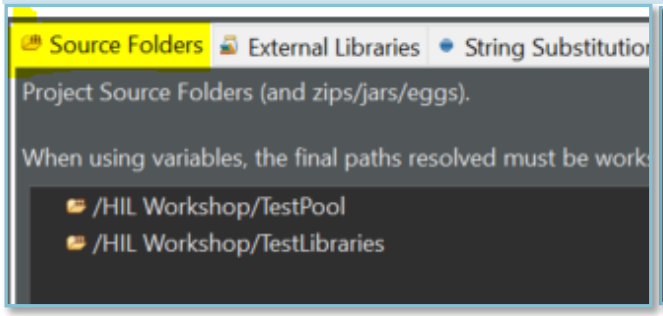
Check that settings for Python interpreter and Python path are correct:

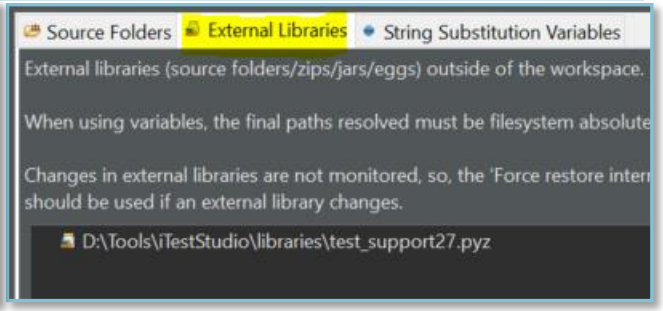
Right-click on the **project name** in PyDev Package Explorer, select „**Properties**“

→ PyDev – Interpreter/Grammar:
select your Python27 interpreter and apply

→ PyDev – PYTHONPATH

- **TestPool** and **TestLibraries** should be Source Folders
- **test_support27.pyz** should be listed in External Libraries





5

Try to run hello_test.py – no error should occur

iTestStudio

iTestStudio

New Project

With **File → New Project** a new project can be created.

All necessary folders and sub-folders will be created in the project folder:	
Python	→ TestPool → TestLibraries
TestReports	→ SingleTestResults
TestStructure	→ TestSequences

Furthermore, a hil_project.ini settings file will be created in the project folder where these paths are configured

Exercise #1: Create a new project

Open iTestStudio and create a new project to D:\HIL-Projekte

with the name **new_project**

→ Check the new created folder, subfolder and the .ini file

Exercise #2: Load an existing project

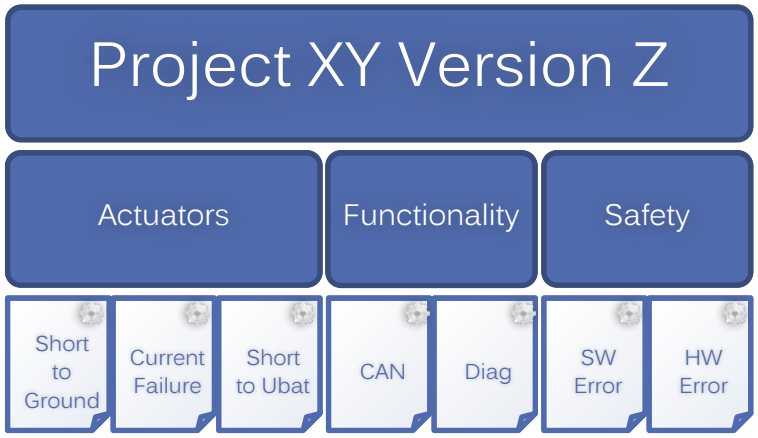
Click the **Select Project** button or use **File → Select Project**

→ Open the **HIL-Workshop** project folder you have copied before

Now all fields should be active and test series can be selected

Explanation: test series / test sequence / test script

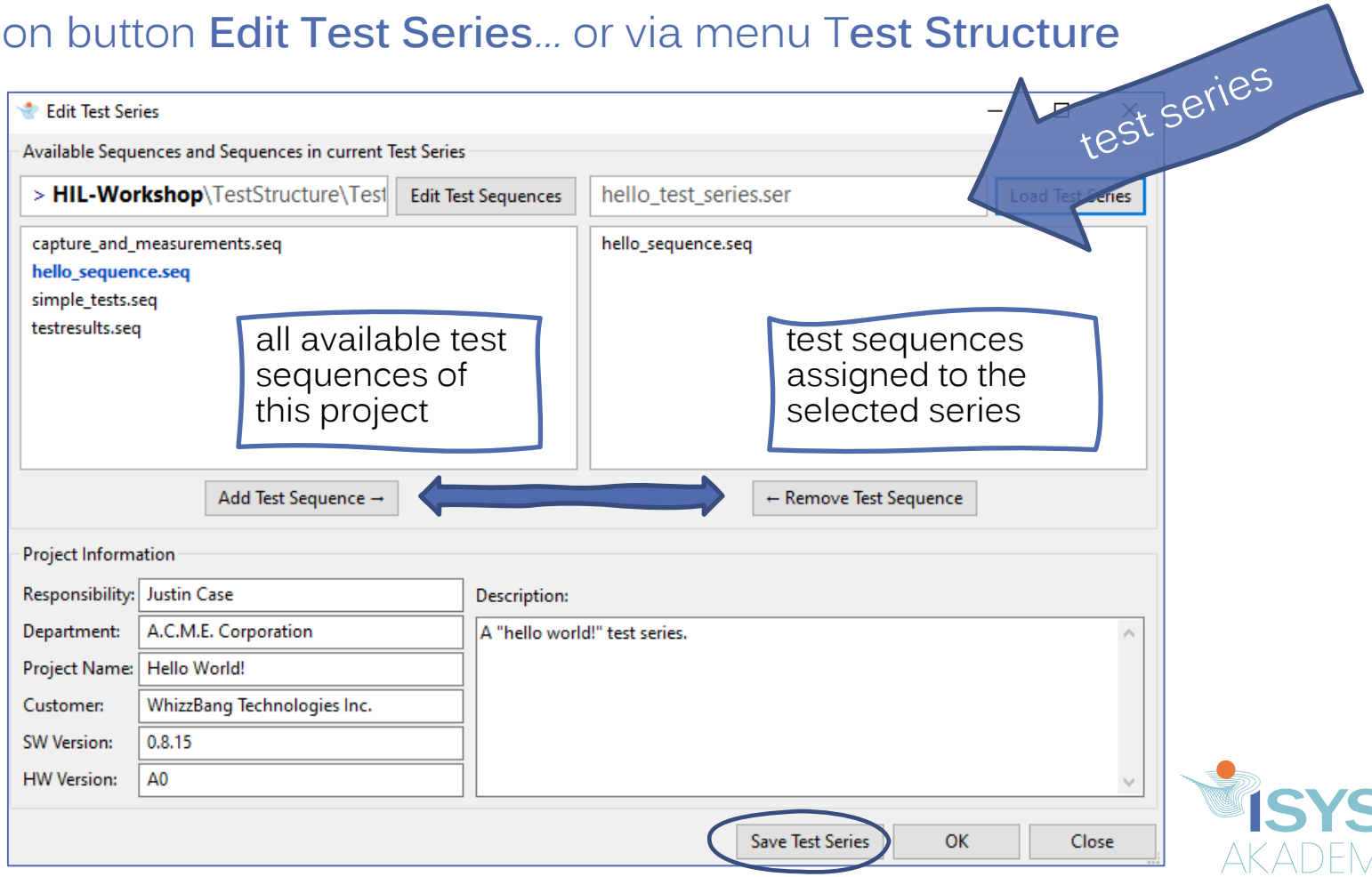
- The project can contain any number of **series**
- To run tests you have to select one **test series** (.ser)
- A test series can contain any number of **sequences** (.seq)
- Inside each sequence any number of **testscripts** can exist



iTestStudio

Create a test series

Click on button **Edit Test Series**... or via menu T**est Structure**



Exercise #3: Create a test series

Create a test series in the HIL-Workshop project with the following sequences:

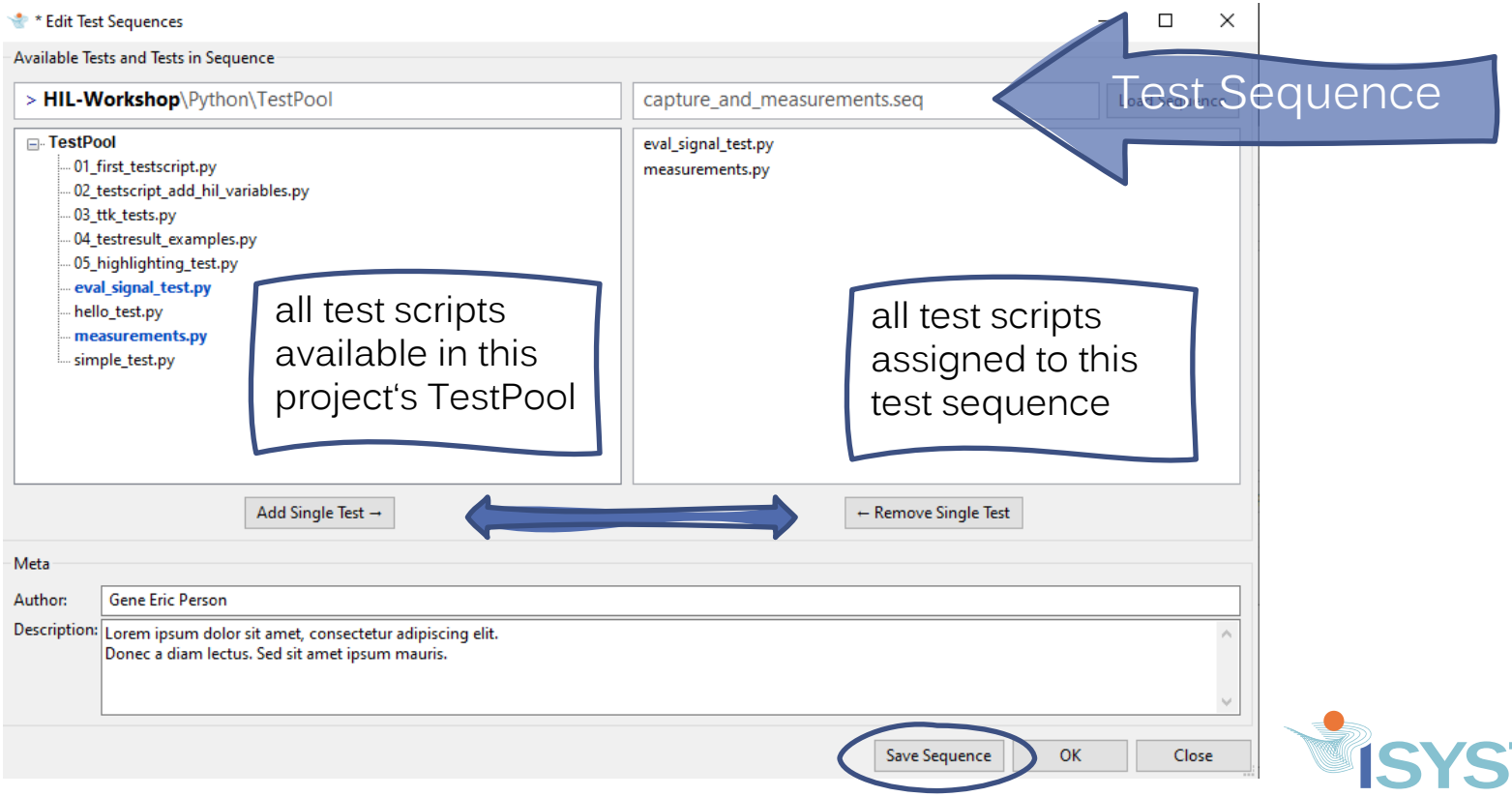
- simple_tests.seq
- hello_sequence.seq
- testresults.seq

Save this test series as **workshop_testserie.ser**

iTestStudio

Create a test sequence

Click [Edit Test Sequences] inside **Edit Test Series** or via the **Test Structure** menu



Exercise #4: Create a test sequence

Add a new test sequence with the name **workshop_testsequence**.

Add following test scripts to the sequence:

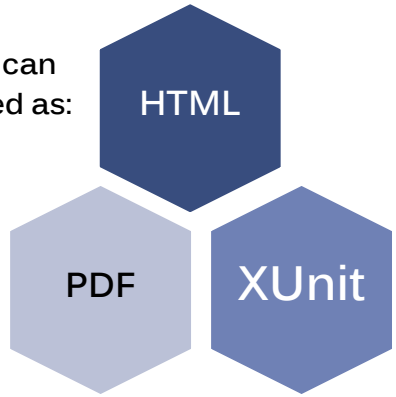
- 01_first_testscript.py
- 02_testscript_add_hil_variables.py
- 03_ttk_tests.py
- 04_testresult_examples.py

Add the sequence to your new test series **workshop_testserie.ser**

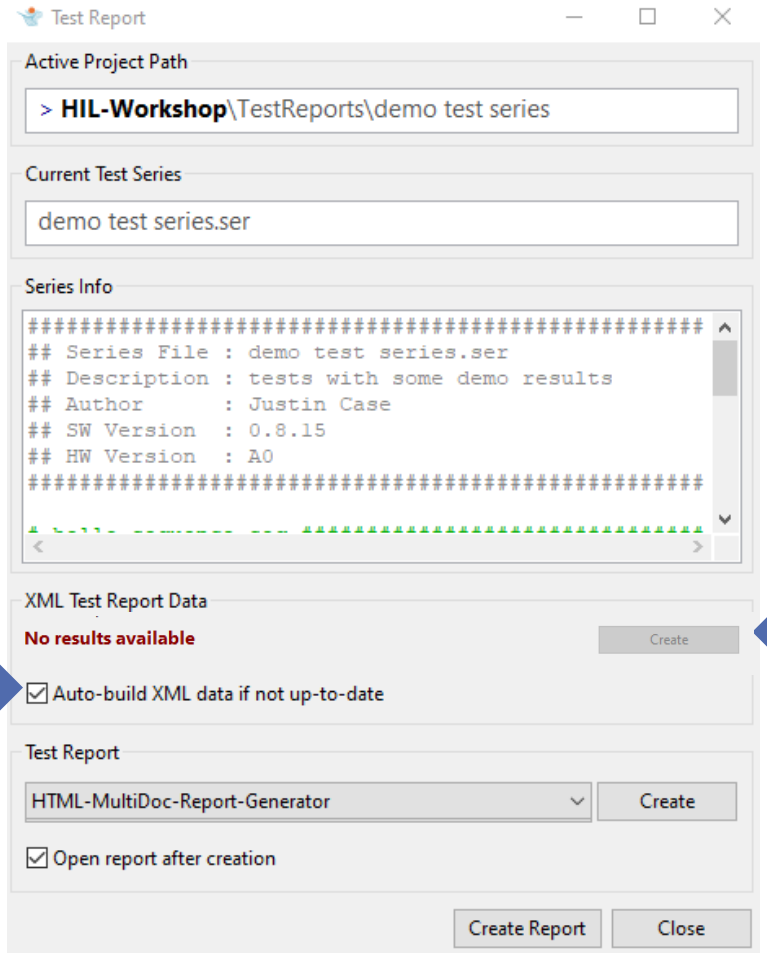
iTestStudio

Generate a Test Report

Testreports can be generated as:



Generate Report...



automatically update report if results changed

create or update the test report

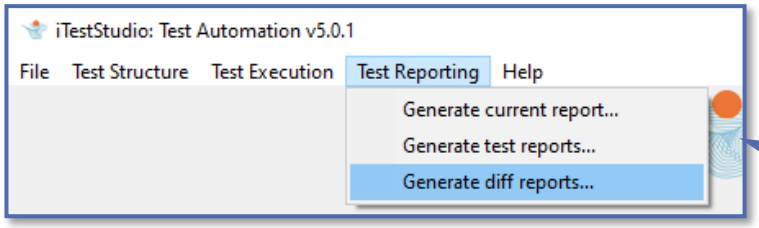
Select if report shall be automatically opened

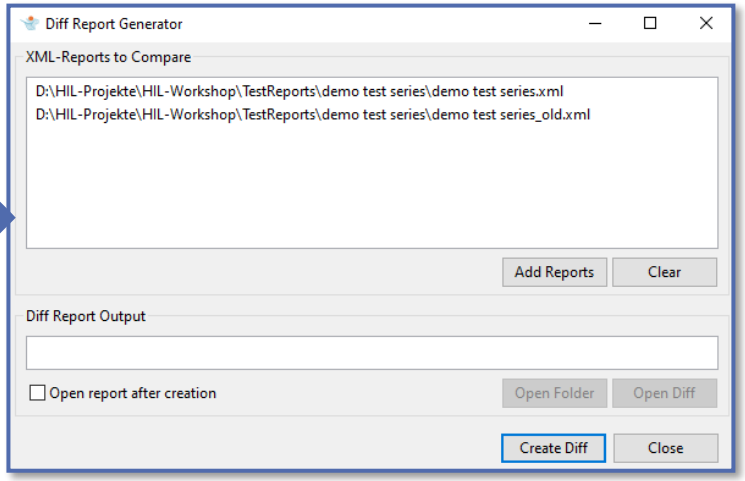
Select format of report output

iTestStudio

Create a Diff-Report

With **Add Reports** multiple xml reports can be inserted to compare the results



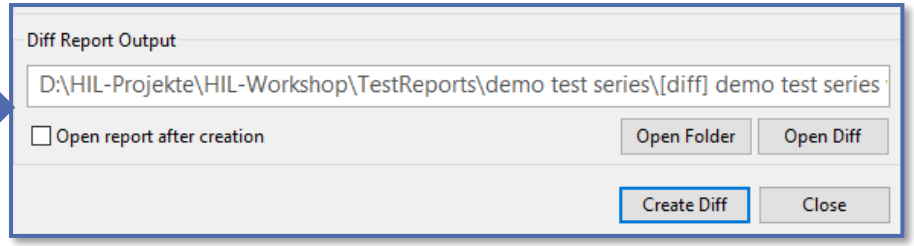


HINT

Diff Report Output has not to be filled out!

It will show the path of the created Diff-Folder

After *Create Diff* was selected the Diff can be shown with *Open Diff*



iTestStudio

Diff Report

	demo test series					demo test series				
Test Sequence	**INFO**	**ERROR**	**Passed**	**Failed**	**Passed [%]**	**INFO**	**ERROR**	**Passed**	**Failed**	**Passed [%]**
hello sequence	0	0	2	1	66.7 %	0	0	2	1	66.7 %
simple tests	0	2	0	4	0.0 %	0	2	0	4	0.0 %
testresults	0	1	0	2	0.0 %	0	1	0	2	0.0 %
capture and measurements	0	3	1	11	8.33 %	0	4	0	8	0.0 %
Overall	**0**	**6**	**3**	**18**	**14.3 %**	**0**	**7**	**2**	**15**	**11.8 %**
	demo test series					demo test series				

Overview:

Different %-values are highlighted

Script:

Different verdicts (PASSED, FAILED) are highlighted

Test Steps:

		demo test series		demo test series	
#	**No.**	**Description**	**Verdict**	**Description**	**Verdict**
1	1	Hello World! **Δt:** 1.913s	PASSED	Hello World! **Δt:** 2.055s	PASSED
2	2	Check nominal supply voltage Expected value: 14.0 V Current value: 0.0 V (CV_mon) Relative Tolerance: +/- 0.05 **Δt:** 1.913s	FAILED	Check nominal supply voltage Expected value: 14.0 V Current value: 0.0 V (CV_mon) Relative Tolerance: +/- 0.05 **Δt:** 2.056s	FAILED
3	3	Current draw in idle should be below 2A 0.0 A (CC_mon) > 2.0 A **Δt:** 1.914s	FAILED	Current draw in idle should be below 2A 0.0 A (CC_mon) < 2.0 A **Δt:** 2.057s	PASSED
		demo test series		demo test series	



HINT

This only works if test steps inside the scripts are equal!

Exercise #5: Create a diff-report

Compare two reports by creating a diff-report:

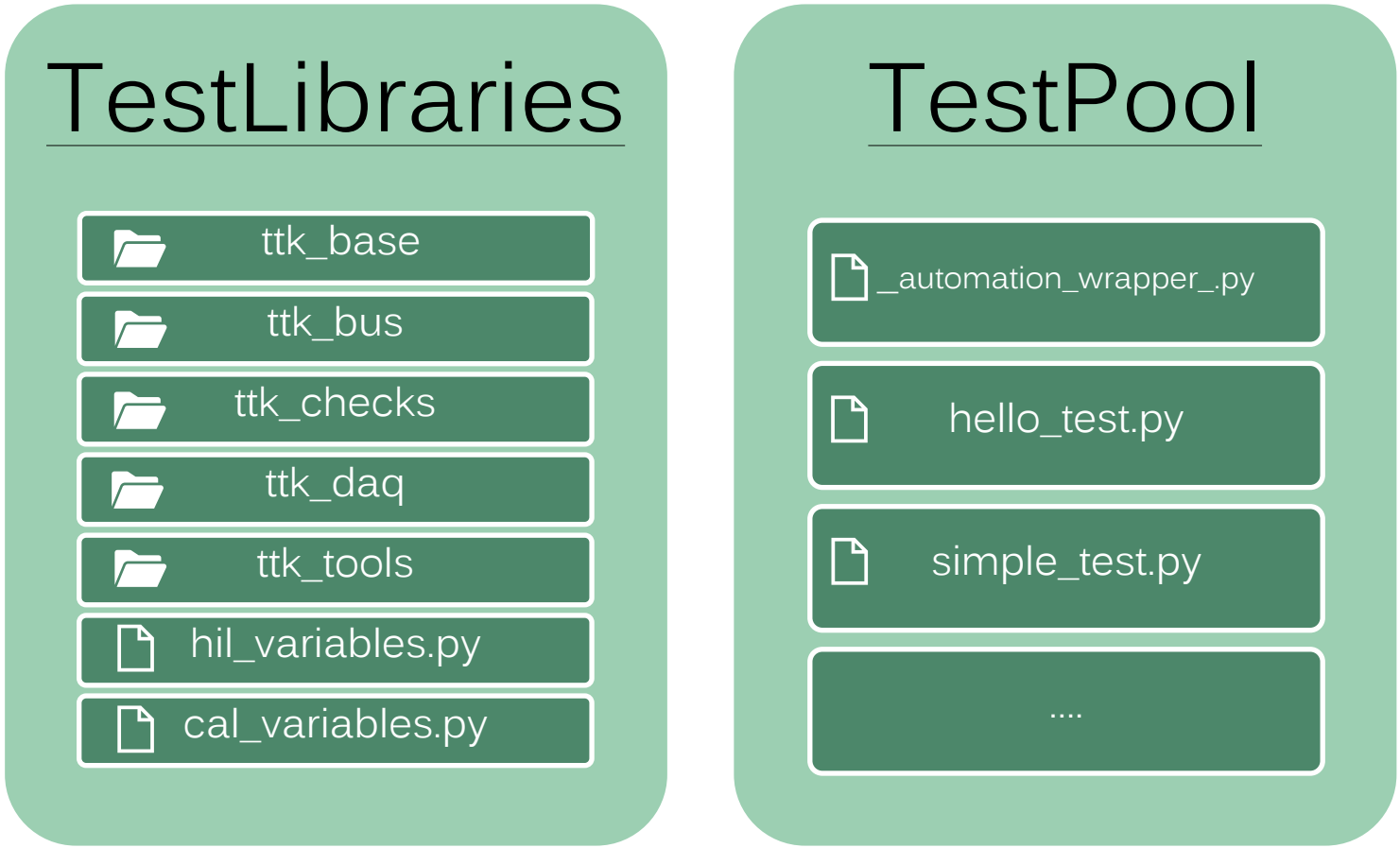
D:\HIL-Projekte\HIL-Workshop\TestReports\hello_test_series

- hello_test_series_01.xml
- hello_test_series_02.xml

TestEnvironment

Structure of Project

Project: HIL-Workshop



_automation_wrapper_.py

```
offline stubs – very important!
```



Useful to check the following parts before running a script on the testsystem:

- syntax, report output, control flow

```
def __setup__()
```

- startup middleware (e.g. GAMMA V)

```
def __breakdown__()
```

- cleanup after test is finished

```
def __getHil__()
```

- to get all HiL-Variables

Test Scripts

```
from _automation_wrapper_ import TestEnv
testenv = TestEnv()

try:
    # ########################################
    testenv.setup()
    # ########################################
finally:
    # ########################################
    testenv.breakdown()
    # ########################################

print "Done."
```

```
def setup(self):
    """ Test Setup: initialize first_testscript environment. """
    # python 2.x!
    TestEnvironmentBase.setup(self)

    # Note: You might want to setup the real-time environment/ControlDesk
    #       connection here during setup, as it will be probably be used
    #       in all first_testscript scripts

    # Gamma V specific handling

    datenmodell_path = os.path.join(_base_project_path, "Settings", "Datenmodell
    config_file = os.path.join(datenmodell_path, "Windows-PC", "Winconfig.xml")

    print("# starting Gamma V service:")
    # try to connect to gamma service
    print("--> try to connect to Gamma service...",)
    try:
        #self.__connectGamma(system_file, config_file)
        self.gamma_service = gamma_api.Gamma(
            system_file          = None,
```

```
def breakdown(self, ecu_shutdown=True):
    """ Test breakdown: cleanup & housekeeping. """
    # restore previous values
    if self.hil:
        self.hil.resetAll()

    if ecu_shutdown and self.hil:
        self.shutdownECU()

    # the base's breakdown() will handle, among other things, storing of
    # the current first_testscript results
    TestEnvironmentBase.breakdown(self)

    # cleanup internal references
    # (typically those were defined during setup)
    self.daq        = None
    self.hil        = None
```

In every script the **_automation_wrapper_.py** has to be called and initialized.

HINT The test steps should always be run inside a try-finally block. In case an error occurs during the test, testenv.breakdown() will run and clean up the test environment before the next test run starts.

Exercise #6:
Changes in _automation_wrapper_

Open the **_automation_wrapper_.py** and make some changes

1. Inside **setup()**
   e.g. add a print „changes in setup“ or what ever you want

2. Inside **breakdown()**
   e.g. add a print „changes in breakdown“ or what ever you want

3. Open **01_first_testscript.py**, run it with your changes from 1. and 2. and check the output.

4. Add startupECU() to the testscript and run the script

Test Scripts

Working with Variables

Access to HiL variables

HiL variables → i.e. values/parameters on real-time system

in this case all middleware variables are inside the HiL variables container: **hil_variables.py**

In a test script:

```
from _automation_wrapper_ import TestEnv
testenv = TestEnv()

try:
    # ######################################
    testenv.setup()
    # ######################################

    hil = testenv.getHil()
```

Now you can read and set all variables inside the **hil_variables.py** container:

```
hil.<variable_name>.set(<value>)
hil.<variable_name>.get()
hil.<variable_name>.reset()
hil.<variable_name>.info()
hil.<variable_name>.getState()
hil.<variable_name>.setState("<state>")
```

Exercise #7: Access to hil variables

Open 01_first_testscript.py

1. Add a call to get hil variables from testenv into the testscript
2. Read out current value of hil variable **clamp_30**
3. Set **clamp_30** to 0 and read out again
4. Read info of **clamp_30**
5. Set **clamp_30** state to „on“ and read out the state again
6. Try to reset **clamp_30**
7. Read the **motor_temp**, set to 40 and then reset

Log every step
→ use print for the
get and info functions

Add HiL Variables

HiL variables require the following parameters:

- context → reference to source (e.g. a gamma api reference)
- identifier → variable identifier (e.g. name, details depend on context)

optional parameters:

- unit → unit as a string
- alias → alias name (otherwise a part of the identifier will be used)
- descr → description of the variable
- lookup → lookup dictionary for states `{value: "state name"}`
- default → value used for reset
- resettable → True (default) or False. If False, variable will not be reset
- fmt → format string used for value e.g. `"%.2f"` or `"hex"`

Exercise #8.1: add hil variables

Part 1: Open **hil_variables.py** in folder **TestLibraries** and add the following variables:

- Variable to read out the current pressure: **current_pressure**
  identifier: „iSyst:IOUserInterface.IO_Display.current_pressure“
  unit: ‚bar‘
- Variable to activate Error Injection: **ei_pressure_enable**
  identifier: „iSyst:IOUserInterface.IO_Parameter.ei_pressure_enable“
  lookup: 1 = active, 0 = inactive
  default: 0
- Variable to set an override pressure value: **ei_pressure_bar**
  identifier: „iSyst:IOUserInterface.IO_Parameter.ei_pressure_bar“
  unit: ‚bar‘
  descr: ‚pressure in bar‘

Exercise #8.2: add hil variables

Part 2: run testscript **02_testscript_add_hil_variables.py** and check the output

1. Add **alias** „pressure“ to the **current_pressure** variable and run again
2. Change the lookup table entry of **ei_pressure_enable** from 1 = active to 2 = active
3. Log **alias** of **ei_pressure_bar** to the testscript

Check the new output of the testscript

Test Scripts

Working with TestToolkit

TestToolkit

basic functions (ttk_checks)

checkTolerance(

current_value,

rated_value,

rel_pos = 0,

abs_pos = 0,

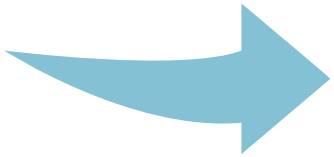
rel_neg = None,

abs_neg = None,

descr = „“,

format = None

)



```
# initial temperature - check tolerance
hil.motor_temp.set(20.0)
time.sleep(0.200)
excepted_value = 20.0

descr, verdict = basic_tests.checkTolerance(
    hil.motor_temp, excepted_value, 0, 0.1,
    descr="Check initial motor temperature"
    )
```

returns a single [<description>, <verdict>] test result entry.

TestToolkit

basic functions (ttk_checks)

checkStatus(
current_status,
nominal_status,
equal = True,
descr = „“,
format = None
)

returns a single [<description>, <verdict>] test result entry.

checkRange(
value,
min_value,
max_value,
descr = „“,
format = None
)

returns a single [<description>, <verdict>] test result entry.

TestToolkit

basic functions (ttk_checks)

Defined values → list of scalars to be searched in

compare(
left_value,
operator,
right_value,
abs_tol = 0,
descr = „“,
format = None
)

returns a single [<description>, <verdict>] test result entry.

Operator (as a string):
==, <, >, <=, >=, !=

contains(
defined_values,
current_value,
mode=„all“,
descr = „“,
format = None
)

returns a single [<description>, <verdict>] test result entry.

mode „all“, „any“ or „none“/“not“

ISYST AKADEMIE

Exercise #9: TTk Functions

Open **03_ttk_tests.py**

1. Run the test script and check output of checkTolerance()

2. Add checkStatus()-function:
   status of **clamp_30** shall not be 0, add a suitable description

3. Add checkRange()-function
   the **motor_temp** shall be between 15°C and 25°C, add a suitable description

4. Add compare()-function
   the **cv_default** shall be larger than at least 10V, add a suitable description

5. Add contains()-function
   the **clamp_15** output shall be either 0 or 1

Test Scripts

Working with Test Results

Test Results

description, verdict

To add testresults to the testreport output, you have to work with testenv.getResults()

```
from _automation_wrapper_ import TestEnv
testenv = TestEnv()

try:
    # ########################################
    testenv.setup()
    # ########################################
    testresult = testenv.getResults()
    # ########################################
```

To add a result to the output you have to add a list containing a description and verdict to the testresult

`['<description text>', '<VERDICT>']` →

```
testresult.append([description, verdict])
testresult += [[description, verdict]]
```

Verdict can be **PASSED**, **FAILED** and **ERROR**

Other verdicts: **INFO**, **TODO** (or even a blank verdict)

Exercise #10: Test Results

Open **04_testresults_example.py**

1. Run the testscript in Eclipse and show the output in the console.

2. Add the testscript **04_testresults_example.py** to the sequence **workshop_testsequence.seq** (iTestStudio) and run this.
   Show the output in the testreport.

3. Add further results with the verdict PASSED, FAILED, ERROR, TODO and let it run again, also create a new report

4. Add a result without a verdict. Run test again – what is the difference between the result entry with verdict and without verdict?

Test Results

TTk Results

TTk functions for evaluating data return a single result as needed for the testresult:

→ **returns** a single [<description>, <verdict>] test result entry.

Read out result, then add the results into testresults

```
descr, verdict = basic_tests.checkTolerance(
    hil.motor_temp, excepted_value, 0, 0.1,
    descr="Check initial motor temperature"
    )

testresult.append([descr, verdict])
```

Add ttk-function output to test results via testresult.append()

```
testresult.append(
    basic_tests.checkTolerance(
        hil.motor_temp, excepted_value, 0, 0.1,
        descr="Check initial motor temperature"
        )
    )
```

Exercise #11: TestToolkit Test Results

Open **03_ttk_tests.py**

1. Change the script, so that all your TTk results appear in the testreport
2. Add this script to your created **workshop_testsequence** and run the tests

Test Results

Result Folder Structure

Test Result

Kind of structure

To store the testresults you have two storage options:

- container
- classic

The newer “container“ structure is active by default starting with iTestStudio 5.0.0. However, the classic structure can be enabled on a per-project basis.

Related options in hil_project.ini

```
[RESULTS]
# container or classic
storage_structure = container
```

Test Results

Classic Structure

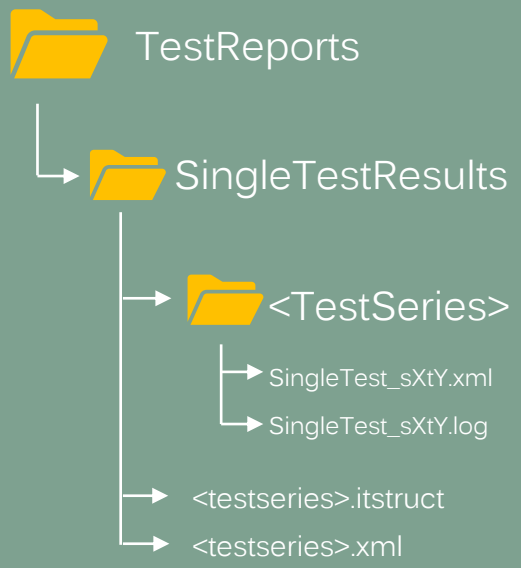
After creating a **new project** with iTestStudio:



results from test runs **outside** of a defined test series



results from test runs from a defined **test series**



Exercise #12: Test Result Classic

Preparation

Open Folder D:\HIL-Projekte\HIL-Workshop\TestReports
→ Delete all files and folders except the **SingleTestResults** Folder

Open Folder D:\HIL-Projekte\HIL-Workshop\TestReports\SingleTestResults
→ Delete all files and folders

Change the storage_structure in the hil_project.ini to classic

1. Open 03_ttk_tests.py and run the script in Eclipse → show in the SingleTestResults

2. Open iTestStudio – select your workshop_testserie.ser
   - Set report generating to „HTML Report Generator“ and run tests → show in the report folder
   - Set report generating to „HTML-Multi-Doc-Report-Generator“ and create new report (without run) → show in the report folder

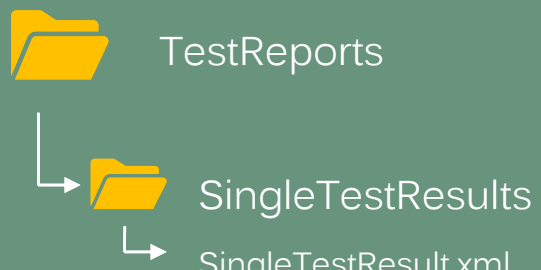
Test Results

Container Structure

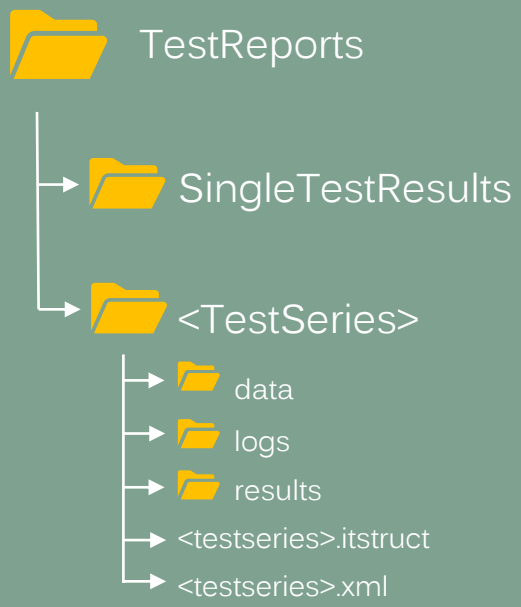
After creating a **new project** with iTestStudio:



results from test runs **outside** of a defined test series



results from test runs from a defined **test series**



Exercise #13: Test Result Container

Preparation

Open Folder D:\HIL-Projekte\HIL-Workshop\TestReports
→ Delete all files and folders except the **SingleTestResults** Folder

Open Folder D:\HIL-Projekte\HIL-Workshop\TestReports\SingleTestResults
→ Delete all files and folders

Change the storage_structure in the hil_project.ini to container

1. Open 03_ttk_tests.py and run the script in eclipse → show into the SingleTestResults

2. Open iTestStudio – select your workshop_testserie.ser
   - Set report generating to „HTML Report Generator“ and run tests → show into the report folder

   - Set report generating to „HTML-Multi-Doc-Report-Generator“ and create new report (without run) → show into the report folder

Test Results

XML Results

Test Results

SingleTestResult – XML meta data

- After every test run – outside or inside a testseries – a result XML will be created
- For every script, a separate xml result will exist

- Every xml file contains the following meta data

```
<meta>
    <title>03 ttk tests</title>
    <description>a test to try out some ttk b
    <author>Neumann</author>
    <date>04.03.2021</date>
    <version>1.0</version>
    <last_revision>initial</last_revision>
    <starttime>2021-03-08T09:16:46</starttime
    <endtime>2021-03-08T09:16:48</endtime>
    <custom_meta_name>Custom Meta Value</cust
    <hil_computername>[redacted]</hil_computer
    <hil_project_path>D:\HIL-Projekte\HIL-Wor
    <hil_username>An3Neumann</hil_username>
</meta>
```

```
#*************************************************************
# -*- coding: latin1 -*-
# File     : 03_ttk_tests.py
# Task     : a test to try out some ttk basic functions
#
# Author   : A. Neumann
# Date     : 04.03.2021
# Copyright 2021 iSyst Intelligente Systeme GmbH
#
#*************************************************************
#******************************** Version ********************
#*************************************************************
# Rev. | Date       | Name     | Description
#-------------------------------------------------------------
# 1.0  | 04.03.2021 | Neumann  | initial
#*************************************************************
```

Test Result

SingleTestResult – XML results

- Every test step with description and verdict is stored in the xml file
- Furthermore, the delta time for every step is stored
- Information for objects (png/jpg) and testcase ids are stored there too (=> iTestStudio Advanced Level)

```
<results>
    <result>
        <description>
        Check initial motor temperature
        Expected value: 20.0 °C (motor_temp)
        Current value:  20.0 °C
        Absolute Tolerance:  +/- 0.1 °C
        </description>
        <verdict>PASSED</verdict>
        <deltatime>2.297</deltatime>
    </result>
</results>
```

```
descr, verdict = basic_tests.checkTolerance(
    current_value = hil.motor_temp,
    excepted_value = 20.0,
    rel_pos = 0,
    abs_pos = 0.1,
    descr="Check initial motor temperature"
    )

testresult.append([descr, verdict])
```

Exercise #14: Test Result XML

SingleTestResults

Go to ...\HIL-Workshop\TestReports\SingleTestResults\ and open **SingleTestManual - 03_ttk_tests.xml** and show the xml

Open **03_ttk_tests.py**. After every change run script again (in Eclipse) and show the changes in the xml

1. Change the description (Task) in the header of the script
2. Add a further revision below 1.0 with current date, your name and a short description
3. Add a further testresult at the begin of the testrun

   description = „Start of the testscript for TestToolkit functions“

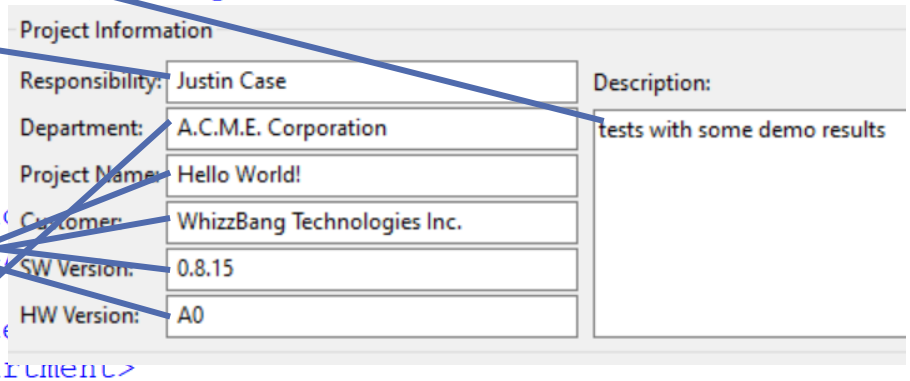
   verdict = „INFO“
4. Delete the verdict „INFO“

Test Results

TestSerie – XML meta data

The result XML for a test series contains **the meta data of the series**, these entries are defined in the series‘ project information in iTestStudio

```
<testreport version="1.0" source="demo test series.ser" xmlns:iT
    <meta>
        <title />
        <description>tests with some demo results</description>
        <author>Justin Case</author>
        <date>2017-12-05T18:50:20</date>
        <version />
        <last_revision />
        <hardwareversion>A0</hardwareversion>
        <softwareversion>0.8.15</softwareversi
        <customer>WhizzBang Technologies In
        <project>Hello World!</project>
        <created_on>2021-03-08T10:32:26</creat
        <department>A.C.M.E. Corporation</depar
    </meta>
```

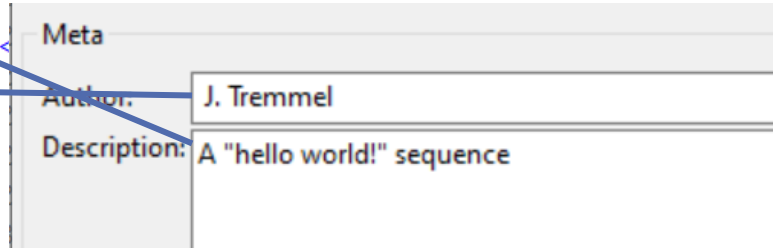


Test Results

TestSerie – XML meta data

The result XML for a test series contains also **meta data for every test sequence**

```
<testsequence source="hello_sequence.seq">
    <meta>
        <title />
        <description>A "hello world!" sequence</
        <author>J. Tremmel</author>
        <date>2015-04-27T13:12:49</date>
        <version />
        <last_revision />
    </meta>
```



Below every test sequence the meta data and test results for every script is stored.

Exercise #15: Test Result XML

Test Series

Go to ...\HIL-Workshop\TestReports\SingleTestResults\ and open the xml of your testseries: **<your test series>.xml**

Open your test series in **iTestStudio**. After every change, run script again and show the changes in the xml

1. Change all project information and add a useful description
2. Change the meta data of one test sequence
3. Open the test report and show the information of series and sequence

Test Structure and Test Execution

From Test Script to Test Result

Exercise #16: Test Script

From Test Script to Test Result

1. Create a new script with the title **pressure_errorinjection.py**, fill out the header with meaningful data

2. The test shall have following steps

    1. Startup the ECU
    2. Read out the **current_pressure**, the value shall be between 2.5 and 4 bar
    3. Activate the error injection by setting **ei_pressure_enable** to 1, write this teststep into a testresult as a „INFO“
    4. Set the **ei_pressure_bar** to 2.4, write this teststep into a testresult as „INFO“
    5. Check that **ei_pressure_bar** and **current_pressure** are equal
    6. Deactivate the error injection by setting **ei_pressure_enable** to 0, write this step into a testresult as a „INFO“
    7. Read out the **current_pressure**, the value shall be between 2.5 and 4 bar

3. Create a second check, with **ei_pressure_bar** set to 4.1. – try to solve this with a for-loop

Exercise #17: Test Series & Sequence

From Test Script to Test Result

1. Create a new test series with the name **TS_Workshop_Basic**
   → fill out the project information and add a description

2. Add a test sequence with the name **pressure_test_sequence**
   → fill out the meta data and description

3. Add your created test **pressure_errorinjection.py** to the sequence

4. Run the test series and create a HTML as well as a HTML-Multi-Doc report

5. Add the current date to the file name of the created xml test report

6. Run the test series again, then create a diff report with the previous and the new xml report



ISYST AKADEMIE

Thank you for your attention!